Product Tagging System - How It Works

URL ingestion vs Image ingestion - architecture, tags, and differences

Component	Location
Scraper (URL flow)	services/product_link_scraper.py - scrape_clothing_product()
Gemini vision (new, URL flow)	services/gemini_service.py - extract_product_metadata_from_evidence()
Gemini vision (legacy, image flow)	services/gemini_service.py - extract_product_metadata()
Reconciliation / tag engine	services/product_metadata.py - resolve_metadata()
Legacy heuristic fallback	services/gemini_service.py - infer_metadata_locally()
API views	api/views_upload.py - AddProductLinkView / UploadProductView / ApproveProductView
Storage model	api/models.py - WardrobeItem
Vision model	gemini-3.6-flash (settings.GEMINI_VISION_MODEL)

1. Overview: there is no single "tags" field

In DripCheck, "product tags" are not one blob of strings. They are typed metadata columns on the WardrobeItem model (api/models.py). Every attribute is a first-class field, most with a fixed vocabulary (Django TextChoices):

- **JSON arrays (free tag lists):** *occasion_type* (e.g. Casual, Business, Formal), *style_tags* (e.g. Minimalist, Streetwear, Classic), *mood_tags* (e.g. Comfy, Elegant)
- **Scalar enum fields:** *category* (Top/Bottom/Footwear/Layer/Accessory), *pattern* (Solid/Stripes/Checks/Graphic/Floral/Abstract), *fit* (Slim/Regular/Relaxed/Oversized/Cropped/Baggy/Tapered), *season* (Summer/Winter/Monsoon/All-season), *color_family* (Neutral/Earth/Dark/Bold/Pastel/Warm), *formality_level* (1-10)
- **Free-text attributes:** *primary_color*, *secondary_color*, *subcategory*, *material*, *brand*

Each of these is generated by its own resolver, then stored together on the same WardrobeItem row. There are two ingestion flows that both end in WardrobeItem, but they generate tags through different pipelines: the URL flow (evidence + vision + reconciliation) and the image-upload flow (legacy Gemini call + heuristics, user approval step).

2. The URL / scraping flow

2.1 What is scraped (services/product_link_scraper.py - scrape_clothing_product)

The scraper validates that the URL is public HTTP(S), fetches the HTML (2 MB cap), parses the tag, meta tags (og:title, og:description, og:image, product:category, product:brand, ...) and every application/ld+json JSON-LD block. From the JSON-LD Product node it extracts name, description, brand, color, material, category, image, additionalProperty (structured attributes) and offers/itemOffered (variants). It then downloads the product image. Everything is preserved in a raw evidence dict: name, description, brand, structured_color, structured_category, structured_material, structured_attributes, variant_data, title, meta, specs_text, image_url, source_url - nothing is discarded.

2.2 What is sent to Gemini (services/gemini_service.py - extract_product_metadata_from_evidence)

One Gemini call per product link. The request contains (a) the actual downloaded product image (base64 inline data, real MIME type detected with PIL) and (b) a text block with all the scraped evidence: name, page title, description, brand, structured color/category/material, structured attributes, variants, specifications text and source URL.

2.3 Model and prompt

Model: gemini-3.6-flash (settings.GEMINI_VISION_MODEL, env-overridable; the previously configured gemini-2.0-flash is retired by Google and returns 404). The prompt instructs the model to inspect the image FIRST and independently - scraped evidence may be wrong or conflict with the photo (e.g. the page lists Maroon while the shirt is black). Visual attributes (primary_color, pattern, fit, garment_type, category, subcategory) must come from what is visible; material should prefer product evidence; occasions and seasons must be conservative; only allowed enum values may be used; each field carries a 0-1 confidence.

2.4 How Gemini returns tags: structured JSON, not free text

The call uses generationConfig.responseMimeType=application/json plus a full OpenAPI responseSchema. The model is therefore forced to return an object with these fields:

Field	Vocabulary	Used for
garment_type	free text (e.g. shirt)	internal, supports category/occasion reasoning
category	Top | Bottom | Footwear | Layer | Accessory	DB field category
subcategory	free text (sanitized)	DB field subcategory
primary_color / secondary_colors	free text	DB fields primary_color / secondary_color

Field	Vocabulary	Used for
pattern	Solid | Stripes | Checks | Graphic | Floral | Abstract	DB field pattern
fit	Slim | Regular | Relaxed | Oversized | Cropped | Baggy | Tapered	DB field fit
material	free text (e.g. Cotton, Linen/Cotton)	DB field material
sleeve	free text (Short/Long/Sleeveless)	internal (season reasoning)
occasion_type	Casual | Formal | Business | Party | Gym | Date Night | Weekend	DB field occasion_type
season	Summer | Winter | Monsoon | All-season	DB field season
formality_level	integer 1-10	DB field formality_level
style_tags	16 predefined tags (Minimalist, Streetwear, Classic, ...)	DB field style_tags
mood_tags	free strings (max 3)	DB field mood_tags
confidence	0-1 per field	internal - gates whether vision values are trusted

2.5 Tag generation is NOT raw AI output: reconciliation (services/product_metadata.py - resolve_metadata)

Gemini output is treated as one evidence source. resolve_metadata() re-derives every field with a fixed priority and records provenance (source per field) and conflicts (page data vs image).

• **Color:** vision+title vs structured JSON-LD. Vision wins if it agrees with title or structured data, or if its confidence >= 0.6; title beats structured when vision is absent; conflicts are logged internally (e.g. title=Black, JSON-LD=Maroon, image=Black -> Black).

• **Fit:** title/description/specs first (slim, skinny, fitted, tailored, regular, relaxed, loose, oversized, boxy, cropped...), then vision with confidence >= 0.6, else neutral Regular at low confidence (the DB schema has no "Unknown" fit).

• **Material:** specs -> JSON-LD structured material -> description -> title -> vision (confidence >= 0.6) -> null. Percentages are parsed (70% Linen / 30% Cotton -> Linen/Cotton). Material is never fabricated from the garment type.

• **Occasion:** explicit text keywords (business/office -> Business; formal -> Formal+Business; party -> Party+Date Night; beach/vacation/hawaiian -> Casual+Weekend; gym -> Gym; smart casual -> Business+Casual), then a garment-type table (suit/blazer -> Formal+Business; dress shirt -> Business+Casual; shirt/polo/tee -> Casual only), then vision (confidence >= 0.6) when no text/garment signal exists, then conservative ['Casual']. The old blanket rule "shirt -> Casual + Business + Date Night" no longer exists in this path.

• **Season:** explicit terms (summer/winter/monsoon/rain), material clues (wool/cashmere/linen), garment construction (sweater/shorts/tank), sleeve length, text clues (breathable/cozy). All-season only as the schema-neutral fallback.

• **Category/subcategory:** structured JSON-LD category -> title/description rules (overshirt -> Layer/Overshirt, dress shirt -> Top/Shirt, polo -> Top/Polo, blazer -> Layer/Blazer, sneaker -> Footwear/Sneaker, ...) -> vision -> keyword fallback.

• **Style/mood tags:** keyword map (minimal -> Minimalist, street -> Streetwear, classic -> Classic, ...) plus validated Gemini style_tags (max 5) and mood_tags (max 3).

• **Brand:** structured data only - Gemini is never asked to invent a brand.

2.6 Storage

api/views_upload.py - build_wardrobe_item_payload() assembles the WardrobeItem dict from the reconciled metadata; WardrobeItemSerializer (api/serializers.py) validates the Django TextChoices and saves the row. The URL flow saves the item directly (no approval step). Response: {success, message, product, source_url} - HTTP 201.

Exact flow:

```
Product URL -> AddProductLinkView.post() (api/views_upload.py) -> scrape_clothing_product()
(product_link_scraper.py: validate -> fetch HTML -> parse title/meta/JSON-LD -> evidence dict ->
download image) -> extract_product_metadata_from_evidence(image + evidence) (gemini_service.py,
gemini-3.6-flash, structured JSON + confidence) -> resolve_metadata(evidence, vision)
(product_metadata.py: reconciliation, provenance, conflicts) -> build_wardrobe_item_payload() ->
WardrobeItemSerializer.save() -> WardrobeItem row
```

3. The image-upload flow

POST /api/wardrobe/upload-product -> UploadProductView.post() (api/views_upload.py). The user must provide the image AND the base fields name, color, type, category - they are compulsory. The server saves the file to media/temp/, then:

- **1) Image enhancement:** gemini_service.generate_ecommerce_image() - Nano Banana (gemini-2.5-flash-image) produces a background-removed e-commerce shot. If it fails, the original is copied (fallback_used=True).
- **2) Metadata/tag extraction:** the LEGACY call gemini_service.extract_product_metadata(image_bytes, name, color, type, category) - same vision model (gemini-3.6-flash) but a prompt-based free-form JSON call: no evidence dict, no responseSchema, no confidence, 8-second timeout. It requests the keys secondary_color, color_family, pattern, fit, occasion_type, season, formality_level, brand, material, style_tags, mood_tags, aesthetic_tone.
- **3) Fallback:** if the Gemini call fails, gemini_service.infer_metadata_locally() runs deterministic heuristics. This legacy engine still contains the broad rules the URL flow removed, e.g. shirt/polo -> occasions ['Casual','Business','Date Night'] and material defaults (shirt -> Cotton, jeans -> Denim, hoodie -> Fleece).
- **4) No storage yet:** the response returns temp image URLs and the metadata to the client for a human approval step.
- **5) Approval:** POST /api/wardrobe/approve-product -> ApproveProductView.post() moves the files from media/temp/ to media/wardrobe/, re-reads the metadata from the request payload (trusting the client) and saves the WardrobeItem.

A third consumer, api/views_avatar.py (GenerateAvatarView), also uses the legacy extract_product_metadata() for outfit-styling context and overrides the color with a PIL pixel-based _detect_garment_color() before the call.

4. Comparison: URL flow vs image flow

Step	Product URL	Product Image
Product data extraction	Scraper: HTML + meta + JSON-LD -> raw evidence dict (name, description, brand, structured color/category/material, attributes, variants, specs, title, image)	None - user must type name, color, type, category
Gemini analysis	extract_product_metadata_from_evidence: image + full evidence, structured output (responseSchema), per-field confidence, ONE call	extract_product_metadata: image + 4 base fields, free-form prompt JSON, no confidence; plus a second AI call (Nano Banana image generation)
Occasion tags	Evidence-based: text keywords -> garment-type table -> vision (conf >= 0.6) -> conservative ['Casual']; no blanket shirt rule	Free-form Gemini occasion_type OR infer_metadata_locally heuristics (still contains blanket shirt -> Casual+Business+Date Night)
Style tags	Keyword map + validated Gemini tags (max 5) + ['Classic'] default	Free-form Gemini style_tags or per-garment heuristics
Color tags	Conflict resolution: vision+title vs structured JSON-LD; conflicts logged; color_family computed	User-typed color; Gemini secondary_color/color_family; avatar flow pixel override
Other attributes	Fit/material/pattern/season from specs/structured/description/vision with provenance; material null when unknown	Gemini free-form or heuristics; material may be invented from garment type (shirt -> Cotton)
Tag normalization	Enums enforced (normalize_vision_result) + evidence rules override AI + DB TextChoices	Prompt-enforced enums + DB TextChoices only
Database storage	Single step, auto-saved, fallback_used reflects Gemini availability	Two-step (upload -> user approval -> save); metadata re-sent from client

5. Will the same black shirt get the same tags via URL and via image?

No - same storage model, different tagging architecture. Both flows write WardrobeItem rows with the same columns, so the output shape is identical, but the semantics are produced by different pipelines:

Aspect	URL flow	Image flow
Primary color	Black (vision + title beat JSON-LD "Maroon"; conflict logged)	Whatever the user typed; no conflict detection
Fit	Slim - from title/description/specs evidence	Free-form Gemini or heuristic; can be Regular default
Material	Cotton only if specs/structured/description say so; null otherwise	Gemini guess or garment-type default (shirt -> Cotton even with zero evidence)
Occasion	['Casual'] for a plain shirt (conservative)	Can become ['Casual','Business','Date Night'] via legacy heuristics
Provenance/conflicts	Recorded per field (sources + confidence)	Not recorded
Human approval	None - auto-saved	Required (approve-product step)

To make both flows share the same tagging architecture, the image flow would need to be migrated to extract_product_metadata_from_evidence() + resolve_metadata() (building a light evidence dict from the user-supplied fields). That is a code change and was intentionally left untouched for this report.

6. Tag vocabularies (api/models.py TextChoices)

Tag group	Allowed values
category	Top, Bottom, Footwear, Layer, Accessory
pattern	Solid, Stripes, Checks, Graphic, Floral, Abstract
fit	Slim, Regular, Relaxed, Oversized, Cropped, Baggy, Tapered
occasion_type	Casual, Formal, Business, Party, Gym, Date Night, Weekend
season	Summer, Winter, Monsoon, All-season
color_family	Neutral, Earth, Dark, Bold, Pastel, Warm
style_tags	Minimalist, Streetwear, Sporty, Vintage, Bohemian, Classic, Business Casual, Y2K, Preppy, Grunge, Monochrome, Techwear, Cottagecore, Bold, Layered, Designer
mood_tags	Free strings, max 3
formality_level	Integer 1 (very casual) to 10 (extremely formal)

7. Key files and functions

File	Functions	Role
services/product_link_scraper.py	scrape_clothing_product, ProductPageParser, find_json_ld_product, extract_structured_attributes, extract_variants, download_product_image, infer_category/infer_subcategory	Scrape URL -> raw evidence + image bytes
services/gemini_service.py	extract_product_metadata_from_evidence (new), extract_product_metadata (legacy), normalize_vision_result, infer_metadata_locally (legacy fallback), generate_ecommerce_image (Nano Banana), _post_generate_content	Gemini REST calls, structured-output schema, enum normalization
services/product_metadata.py	resolve_metadata, _resolve_color, _resolve_fit, _resolve_material, _resolve_occasion, _resolve_season, _resolve_category, _resolve_tags	Evidence-based tag generation, reconciliation, provenance, conflicts

File	Functions	Role
api/views_upload.py	AddProductLinkView (URL flow), UploadProductView (upload step), ApproveProductView (approve step), build_wardrobe_item_payload	HTTP endpoints, payload assembly, storage
api/models.py	WardrobeItem + TextChoices enums	Storage schema and tag vocabulary
api/serializers.py	WardrobeItemSerializer	Validation and persistence
dripcheck_django/settings.py	GEMINI_API_KEY, GEMINI_VISION_MODEL	API key and model config

Report date: August 2026. Documented against the current codebase after the product-link ingestion upgrade (evidence preservation, Gemini vision on downloaded images, structured output, reconciliation, gemini-3.6-flash).